DOUBLE-SCAN

The Intelligent Ledger

MVP Build Plan — 1-Week Sprint

MD Monzurul Islam | Cybersecurity & Blockchain Developer

April 2026

Executive Summary

Double-Scan is a software-only anti-counterfeiting platform that combines Behavioral AI with a Blockchain Oracle to detect cloned QR codes in real time — without requiring any hardware changes. This document provides a complete 1-week MVP build plan that faithfully represents the core product, enabling a demo-ready system that proves the concept and is ready for pilot onboarding.

7 Days	3 Core Modules	$0 Hardware
Build Timeline	Detective / Witness / Bridge	Zero CAPEX Infrastructure

What the MVP Must Prove

The MVP is not a toy — it is a trimmed but faithful version of the full product. Every core mechanism must be present and demonstrable. The MVP must prove:

- A QR code can be registered on-chain (genesis record) with zero hardware
- The AI engine scores scan telemetry in real time (GPS, IP, device, timestamp)
- Impossible Travel is detected and triggers an automatic blacklist
- Trust Triage distinguishes a legitimate scan from a cloned scan in the same city
- The Blockchain Oracle writes only AI-verified events on-chain
- A dashboard shows live fraud alerts and scan heatmaps for manufacturers

MVP Architecture

The MVP preserves the full three-layer architecture of the product: Detective (AI engine), Witness (blockchain ledger), and Bridge (oracle API). What is simplified is scope — not structure.

Layer	Component	Technology	MVP Scope
Detective	Behavioral AI Engine	Node.js (Fastify) + Risk Scorer	Impossible Travel + Trust Triage algorithms
Witness	Layer-2 Ledger	Polygon Mumbai (testnet)	Minted / Active / Flagged states
Bridge	Oracle / API	REST API + Oracle Signing	Vetted writes only; SHA-256 signed
Data	Storage	PostgreSQL + Redis (session)	Scan metadata + sub-10ms cache
UI	Dashboard	React (Vite)	Manufacturer alerts + scan map

7-Day Sprint Plan

Each day has a clear deliverable. By end of Day 7, you have a fully demo-ready MVP.

Day 1 — Project Setup & Blockchain Foundation

Goal: Working blockchain contract deployed to testnet with product registration.

1. Initialize monorepo with two packages: /backend and /frontend
2. Install: Node.js 20+, Fastify, ethers.js, Hardhat, PostgreSQL, Redis
3. Write Solidity smart contract: ProductRegistry.sol

ProductRegistry.sol must implement:

- mintProduct(productId, metadataHash) — genesis record, emits ProductMinted event
- flagProduct(productId, reason) — called by oracle only, emits ProductFlagged
- getProduct(productId) — returns state: 0=Minted, 1=Active, 2=Flagged
- Ownable modifier: only the oracle wallet can call flagProduct

4. Deploy to Polygon Mumbai testnet via Hardhat: npx hardhat run scripts/deploy.js --network mumbai

5. Save contract address to .env. Create PostgreSQL schema: products table, scans table
6. Deliverable: Contract deployed, address confirmed on PolygonScan Mumbai

File to Create	Purpose
contracts/ProductRegistry.sol	Solidity smart contract
scripts/deploy.js	Hardhat deployment script
db/schema.sql	PostgreSQL table definitions
.env	PRIVATE_KEY, RPC_URL, CONTRACT_ADDRESS

Day 2 — Scan Ingestion API

Goal: POST /scan endpoint that accepts telemetry and stores it in PostgreSQL.

7. Create Fastify server in /backend/src/server.js
8. Build POST /api/scan endpoint — accepts this JSON payload:

Field	Type	Description
productId	string	The QR code's unique product identifier
deviceId	string	SHA-256 fingerprint of scanning device
ipAddress	string	IPv4 or IPv6 of the scanner
gpsLat / gpsLng	float	Coordinates from phone GPS
timestamp	ISO 8601	Exact scan time (UTC)
userAgent	string	Browser/app user agent string

9. Write scan to scans table in PostgreSQL with a UUID scan_id
10. Add Redis caching: store last scan per productId with TTL of 3600s
11. Build GET /api/product/:id — returns product chain state + last 10 scans
12. Build POST /api/product/register — manufacturer registers a new product, mints on chain
13. Deliverable: Postman/curl can register a product and submit a scan successfully

Day 3 — Behavioral AI Engine (The Detective)

Goal: Risk scoring engine that detects Impossible Travel and Trust Triage anomalies.

Create /backend/src/ai/riskEngine.js. This is the core intelligence of Double-Scan.

Algorithm 1: Impossible Travel Detection

- Retrieve last confirmed scan for the productId from Redis cache
- Calculate distance between previous GPS and current GPS (Haversine formula)
- Calculate time delta in hours between the two scans
- Compute velocity = distance / time_delta (km/h)
- If velocity > 900 km/h AND time_delta < 3 hours: IMPOSSIBLE TRAVEL flag
- Risk contribution: +60 points

Algorithm 2: Trust Triage (Same-Location Conflict)

- If same productId scanned by 2+ different deviceIds within 30 minutes: conflict detected
- Score each device: known/registered device = -20 pts; unknown device = +30 pts
- Residential IP (via IP range check) at unusual hour (midnight–5am) = +25 pts
- Medical/enterprise IP subnet pattern = -15 pts
- New device fingerprint never seen before = +20 pts

Risk Score Thresholds

Score Range	Verdict	Action
0–30	TRUSTED	Write to blockchain as Active
31–69	SUSPICIOUS	Flag for review, alert manufacturer
70–100	COUNTERFEIT	Blacklist token, trigger oracle

14. Attach riskEngine to the POST /api/scan handler — every scan is scored before storage
15. Return riskScore, verdict, and reasons array in the API response
16. Deliverable: Simulate 2 scans for same productId from Sydney then Dhaka — verdict: COUNTERFEIT

Day 4 — Blockchain Oracle (The Bridge)

Goal: AI verdicts trigger on-chain state changes via the oracle. Only verified events become canonical.

17. Create /backend/src/oracle/oracleService.js
18. Oracle logic flow:

- If verdict is TRUSTED: call contract.updateState(productId, 1) — sets state to Active
- If verdict is COUNTERFEIT: call contract.flagProduct(productId, reason) — sets state to Flagged

- If verdict is SUSPICIOUS: store in PostgreSQL as pending_review, do not write on-chain yet

19. Sign each oracle transaction with the oracle's private key (stored in .env, never exposed)
20. Add oracle call to POST /api/scan: after risk scoring, invoke oracle if score > 30
21. Store txHash in the scans table for every on-chain event
22. Create GET /api/proof/:scanId — returns txHash + PolygonScan link for audit trail
23. Deliverable: A blacklisted product shows Flagged state on PolygonScan Mumbai

Day 5 — Manufacturer Dashboard (React Frontend)

Goal: A functional dashboard that shows live alerts, scan heatmap, and product registry.

24. Scaffold React app: npm create vite@latest frontend -- --template react
25. Install: axios, react-leaflet (map), recharts (charts), tailwindcss
26. Build 4 pages/components:

Component	What It Shows
Dashboard Overview	Total scans today, flagged count, trusted count, risk score distribution chart
Scan Heatmap	Leaflet map with dot markers colored by verdict (green=trusted, red=flagged)
Alert Feed	Live list of SUSPICIOUS and COUNTERFEIT events with productId, location, timestamp
Product Registry	Table of registered products, their chain state, and a Register New Product form

27. Connect to backend via REST: poll GET /api/dashboard every 5 seconds for live data
28. Add GET /api/dashboard endpoint to backend: returns aggregate stats + recent alerts
29. Deliverable: Dashboard runs at localhost:5173, shows live flag events when simulated

Day 6 — Consumer Verification Portal & Demo Data

Goal: A public-facing page consumers can visit to check a product's authenticity. Seed demo data for the full narrative.

30. Add a public route in the React app: /verify/:productId
31. Page fetches GET /api/product/:id and displays:

- Green banner + checkmark if state is Active/Trusted
- Red warning banner if state is Flagged — 'This product may be counterfeit'
- Product name, manufacturer, registration date, scan count

- Link to blockchain proof (PolygonScan transaction)

32. Seed demo data — create a script: node scripts/seedDemo.js that creates:

- Product A: 'Pharma Batch PH-2024-001' — trusted, 47 scans, all legitimate
- Product B: 'Luxury Watch LW-0099' — flagged via Impossible Travel (Sydney → Dhaka in 2h)
- Product C: 'Medical Supply MS-442' — Trust Triage scenario, 1 hospital scan (trusted) + 1 midnight residential scan (counterfeit)

33. Deliverable: Scanning QR codes for Products A, B, C shows distinct outcomes in the portal

Day 7 — Integration Testing, QR Code Generation & Demo Hardening

Goal: End-to-end flows working reliably. QR codes generated for live scanning. Demo script ready.

34. Install qrcode npm package. Create POST /api/qr/:productId — returns a PNG QR code that encodes the consumer verification URL
35. Print or display QR codes for the 3 demo products
36. Run end-to-end tests for both key scenarios:

Test Scenario	Input	Expected Output
Impossible Travel	Same product scanned Sydney 10:00AM, Dhaka 12:00PM	Verdict: COUNTERFEIT, token blacklisted on-chain
Trust Triage	Same product: hospital iPad 09:00AM + unknown device 03:00AM	Hospital scan: TRUSTED; night scan: COUNTERFEIT
Clean Scan	Known device, registered IP, normal hours	Verdict: TRUSTED, state: Active
Audit Trail	GET /api/proof/:scanId for a flagged scan	Returns txHash + PolygonScan link

37. Fix any integration issues. Verify Redis cache is serving sub-10ms responses
38. Write DEMO.md with step-by-step demo script for investor/client presentations
39. Deliverable: Full live demo ready. One QR code scan triggers the complete pipeline

---

Complete Project File Structure

File / Folder	Purpose
/backend/src/server.js	Main Fastify server entry point
/backend/src/api/scan.js	POST /api/scan handler
/backend/src/api/product.js	Product registration & query routes
/backend/src/ai/riskEngine.js	Behavioral AI: Impossible Travel + Trust Triage
/backend/src/oracle/oracleService.js	Blockchain Oracle: signs and writes state changes
/backend/src/db/postgres.js	PostgreSQL connection + query helpers
/backend/src/db/redis.js	Redis client + session cache helpers
/backend/contracts/ProductRegistry.sol	Solidity smart contract (states + events)
/backend/scripts/deploy.js	Hardhat deploy to Polygon Mumbai
/backend/scripts/seedDemo.js	Seed 3 demo products with scan histories
/backend/db/schema.sql	PostgreSQL: products + scans tables
/frontend/src/pages/Dashboard.jsx	Manufacturer overview: stats + alerts
/frontend/src/pages/Heatmap.jsx	Leaflet scan location map
/frontend/src/pages/Registry.jsx	Product list + registration form
/frontend/src/pages/Verify.jsx	Public consumer verification page
.env	PRIVATE_KEY, DB_URL, REDIS_URL, CONTRACT_ADDRESS
DEMO.md	Step-by-step demo script for presentations

Environment Setup (Day 1 Checklist)

Complete all of these before writing a single line of application code.

Required Accounts & Services

- Alchemy or Infura account — free tier, get Polygon Mumbai RPC URL
- MetaMask wallet — generate a fresh wallet, save private key to .env (NEVER commit this)
- Fund wallet with Mumbai testnet MATIC — use faucet: faucet.polygon.technology
- PolygonScan Mumbai account — free API key for transaction verification

Local Stack

- Node.js 20+ — node -v to verify
- PostgreSQL 15 — brew install postgresql or apt-get install postgresql
- Redis 7 — brew install redis or apt-get install redis
- Hardhat — npm install -g hardhat

What Is Excluded from MVP (and Why)

These items are part of the full product but are deliberately excluded from the 1-week MVP to keep the build focused and shippable.

Full Product Feature	MVP Status	Reason for Exclusion
Ethereum Mainnet deployment	Testnet only (Mumbai)	Cost & complexity; testnet proves the same logic
Device fingerprinting SDK	Simulated via deviceId field	Requires native SDK; API field sufficient for demo
Advanced IP geolocation (commercial)	Basic IP range rules	Paid API not needed to demonstrate triage logic
Manufacturer onboarding flow / auth	Hardcoded test manufacturer	Auth adds days; demo needs one working manufacturer
DSCSA / FMD compliance export	Not included	Regulatory formatting; core detection is what investors need to see
Mobile app (iOS/Android)	Browser-based only	QR scanning in browser is sufficient for pilot demo

---

MVP Success Criteria

The MVP is complete when all of the following are demonstrable in a live session:

#	Criteria	Proof
1	A new product can be registered and minted on-chain in < 30 seconds	PolygonScan tx link
2	Impossible Travel scan scores 70+ and triggers automatic blacklist	Dashboard alert + on-chain Flagged state
3	Trust Triage correctly identifies hospital scan as Trusted (score < 30) and night scan as Counterfeit (score > 70)	API response JSON + dashboard

#	Criteria	Proof
4	Every oracle write produces a retrievable txHash and PolygonScan proof link	GET /api/proof/:scanId response
5	Consumer verification URL correctly displays TRUSTED or FLAGGED for each demo product	Browser demo
6	Redis cache delivers last-scan lookups in under 10ms	Fastify response time logs
7	Dashboard shows live scan map and alert feed updating in real time	Screen share

---

Demo Script for Presentations

Follow this sequence for investor or client demos. Each step takes approximately 2 minutes.

Step 1: Register a Product (2 min)

- Open Manufacturer Dashboard → Registry tab
- Click Register New Product → enter 'Pharma Batch PH-2026-DEMO'
- Show the PolygonScan Mumbai link that appears — product is now on-chain

Step 2: Demonstrate Impossible Travel (3 min)

- Use Postman or the demo seed script to fire scan #1 from Sydney (GPS: -33.87, 151.21) at T+0
- Fire scan #2 from Dhaka (GPS: 23.81, 90.41) at T+2 hours — same productId
- Point to the Dashboard Alert Feed — COUNTERFEIT verdict appears within seconds
- Open PolygonScan — product state is now Flagged
- Scan the consumer QR code — red warning banner appears

Step 3: Demonstrate Trust Triage (3 min)

- Fire the night scan: unknown deviceId, residential IP, 03:00 AM — Risk Score 92
- Fire the hospital scan: registered iPad deviceId, medical subnet IP, 09:00 AM — Risk Score 5
- Dashboard shows: one COUNTERFEIT event and one TRUSTED event for the same product
- Explain: 'The AI doesn't just count conflicts — it evaluates context'

Step 4: Show Audit Trail (1 min)

- Call GET /api/proof/:scanId for the blacklisted scan
- Show the txHash and PolygonScan link — this is the forensic evidence
- Key message: 'This is admissible chain-of-custody evidence for enforcement and liability protection'

Technology Stack Summary

Layer	Technology	Version / Notes
Runtime	Node.js	v20 LTS
API Framework	Fastify	v4 — faster than Express, built-in schema validation
Blockchain	Hardhat + ethers.js	Deploy & interact with Solidity contracts
Smart Contract	Solidity 0.8+	Deployed to Polygon Mumbai testnet
Database	PostgreSQL 15	Scan metadata, product registry, audit log
Cache	Redis 7	Last-scan sessions, sub-10ms lookups
Frontend	React 18 + Vite	Manufacturer dashboard + consumer portal
Map	React-Leaflet	Open-source; no Google Maps API key needed
Charts	Recharts	Risk score distribution, scan volume over time
QR Codes	qrcode (npm)	Server-side PNG generation per product
Hashing	SHA-256 (Node crypto)	Device fingerprinting + oracle signing

---

Post-MVP Roadmap

Once the MVP is validated through a pilot, the following enhancements move the product toward full production readiness.

Phase	Milestone	Description
Phase 2	Mainnet Deployment	Migrate from Polygon Mumbai testnet to Polygon Mainnet; real economic stake for oracle integrity

Phase	Milestone	Description
Phase 2	Manufacturer Auth	JWT-based multi-tenant authentication; each manufacturer sees only their products
Phase 3	Device SDK	Native iOS/Android SDK for true device fingerprinting beyond user-submitted deviceId
Phase 3	Compliance Export	DSCSA (US) and FMD (EU) formatted audit log exports for regulatory submissions
Phase 4	Enterprise Integrations	Webhook push to manufacturer ERP systems (SAP, Oracle) on fraud detection events
Phase 4	ML Risk Model	Replace rule-based scoring with a trained anomaly detection model on real scan data

Double-Scan: The Intelligent Ledger

We don't just verify codes — we verify the truth.

Contact: MD Monzurul Islam | LinkedIn | April 2026